DBMS Assignment 3

Airline Reservation System

1. Functional Dependencies

Key	Functional dependency
Passenger_ID	Passenger_ID → Name, Email, Phone, Passport_No
Email / Passport_No	Each unique value identifies one passenger.
Airport_ID	Airport_ID → Airport_Name, City, Country, Airport_Code
Aircraft_ID	Aircraft_ID → Model, Capacity, Manufacturer
Seat_ID	Seat_ID → Aircraft_ID, Seat_Number, Class
Flight_ID	Flight_ID → Flight_Number, Aircraft_ID, airports, times, Status
Booking_ID	Booking_ID → Passenger_ID, Flight_ID, Seat_ID, Date, Status
Payment_ID	Payment_ID → Booking_ID, Amount, Date, Method, Status

2. Redundancy and Anomalies

Keeping passenger details in every booking would repeat email and passport details. Storing airport names and cities in every flight could repeat the same information. Separate tables reduce this redundancy.

Insertion anomaly: If passenger and flight data were in one table, a passenger could not be added before making a booking. Separate Passenger and Flight tables avoid this.

Update anomaly: If an airport city changed, many duplicated flight rows would need editing. Keeping airport details in Airport allows one update.

Deletion anomaly: Deleting the last booking should not remove the passenger, airport or flight master details. Separate tables protect these records.

3. Normalization

1NF: each field stores one value. 2NF: non-key attributes depend on the whole key. 3NF: airport and aircraft details are stored in their own tables instead of being repeated in Flight.

4. Improvements

Use primary and foreign keys, unique email and passport values, unique airport codes, positive aircraft capacity, valid status checks, and a unique (Flight_ID, Seat_ID) rule. Also check that the seat belongs to the aircraft assigned to the flight; a simple foreign key alone does not enforce this cross-table rule.

5. Conclusion

The seven-table design reduces repeated data and prevents common anomalies. Constraints and transaction checks can improve accuracy further.